Министерство Образования и исследований

Государственный Университет Молдовы

Кафедра информатики

Отчет:

По дисциплине Java Script

Лабораторная работа No3

Tema: **Основы работы с массивами, функциями и объектами в JavaScript**

Выполнил:

Чубару Максим, gr. IA2503

Проверил:

N.Calin

Кишинев, 2026

Цель работы

Целью данной лабораторной работы является изучение основ работы с массивами и функциями в JavaScript, применяя их для обработки и анализа транзакций. В ходе работы студент знакомится с методами итерации массивов, фильтрацией и агрегацией данных, а также с использованием структуры данных Set для извлечения уникальных значений.

Теоретическая часть

Формулировка задачи

В рамках лабораторной работы требовалось:

- Создать массив объектов транзакций с заданными свойствами.
- Реализовать набор функций для анализа и обработки этого массива.
- Продемонстрировать работу каждой функции на практике.

Каждый объект транзакции содержит следующие поля:

- transaction_id — уникальный идентификатор.
- transaction_date — дата транзакции (формат YYYY-MM-DD).
- transaction_amount — сумма транзакции.
- transaction_type — тип (debit или credit).
- transaction_description — описание транзакции.
- merchant_name — название магазина/сервиса.
- card_type — тип карты.

Описание основных этапов работы

В ходе выполнения лабораторной работы были выполнены следующие этапы:

1. Создание файла transactions.js с массивом тестовых транзакций.
2. Создание файла main.js с реализацией всех требуемых функций.
3. Проверка работы каждой функции через вызовы с тестовыми данными.

Практическая часть

Краткое описание особенностей реализации

1. getUniqueTransactionTypes(data)

Функция возвращает уникальные типы транзакций. Реализована с использованием Set — структуры данных, автоматически исключающей дублирующиеся значения.

```
function getUniqueTransactionTypes(data) {
    const array = [];
    for (const group of data) {
        for (const item of group) {
            array.push(item?.transaction_type);
        }
    }
    return new Set(array);
}
```

2. calculateTotalAmount(data)

Вычисляет суммарный объём всех транзакций. Используется вложенный цикл с методом reduce для накопления суммы внутри каждой группы.

```
function calculateTotalAmount(data) {
    let sum = 0;
    for (const group of data) {
        sum += group.reduce((acc, cur) => acc + cur?.transaction_amount, 0);
    }
```

```
    return sum;
}
```

3. getTransactionByType(data, type)

Выводит все транзакции заданного типа (debit или credit). Применяется двойной цикл с условной проверкой.

```
function getTransactionByType(data, type) {
    for (const group of data) {
        for (const item of group) {
            if (item?.transaction_type == type) console.log(item);
        }
    }
}
```

4. getTransactionsInDateRange(data, startDate, endDate)

Возвращает транзакции в указанном диапазоне дат. Для числового сравнения дат используется вспомогательная функция dateToNumber, удаляющая дефисы и преобразующая строку в число.

```
function dateToNumber(str) {
    return Number(str.split('-').join(''));
}
function getTransactionsInDateRange(data, startDate, endDate) {
    for (const group of data) {
        for (const item of group) {
            const d = dateToNumber(item?.transaction_date);
            if (d >= dateToNumber(startDate) && d <= dateToNumber(endDate))
                console.log(item);
        }
    }
}
```

5. getTransactionsByMerchant(data, merchantName)

Возвращает массив транзакций, совершённых у конкретного продавца. Используется метод forEach для обхода группы.

```
function getTransactionsByMerchant(data, merchantName) {
    const arr = [];
    for (const group of data) {
        group.forEach(el => {
            if (el?.merchant_name == merchantName) arr.push(el);
        });
    }
    return arr;
}
```

6. calculateAverageTransactionAmount(data)

Вычисляет среднее значение суммы транзакций. Отслеживается количество транзакций и накопленная сумма.

```
function calculateAverageTransactionAmount(data) {
    let sum = 0, count = 0;
    for (const group of data) {
        for (const item of group) { sum += item?.transaction_amount; count++; }
    }
    return sum / count;
}
```

7. getTransactionsByAmountRange(data, minAmount, maxAmount)

Фильтрует транзакции по диапазону суммы включительно.

```
function getTransactionsByAmountRange(data, minAmount, maxAmount) {
    const arr = [];
    for (const group of data) {
        for (const item of group) {
            if (item?.transaction_amount >= minAmount &&
                item?.transaction_amount <= maxAmount) arr.push(item);
        }
    }
    return arr;
}
```

8. calculateTotalDebitAmount(data)

Суммирует исключительно дебетовые транзакции.

```
function calculateTotalDebitAmount(data) {
    let sum = 0;
    for (const group of data) {
        for (const item of group) {
            if (item?.transaction_type == 'debit') sum += item?.transaction_amount;
        }
    }
    return sum;
}
```

9. findMostTransactionsMonth(data)

Возвращает номер месяца с наибольшим количеством транзакций. Используется массив из 12 счётчиков (по одному на каждый месяц), индексируемый по номеру месяца из даты.

```
function findMostTransactionsMonth(data) {
    const arr = new Array(12).fill(0);
    for (const group of data) {
        for (const item of group) {
            arr[parseInt(item?.transaction_date.split('-')[1], 10) - 1]++;
        }
    }
    return arr.indexOf(Math.max(...arr)) + 1;
}
```

10. findMostDebitTransactionMonth(data)

Аналогична предыдущей функции, но учитывает только дебетовые транзакции.

```
function findMostDebitTransactionMonth(data) {
    const arr = new Array(12).fill(0);
    for (const group of data) {
        for (const item of group) {
            if (item?.transaction_type == 'debit')
                arr[parseInt(item?.transaction_date.split('-')[1], 10) - 1]++;
        }
    }
    return arr.indexOf(Math.max(...arr)) + 1;
}
```

11. mostTransactionTypes(data)

Сравнивает количество дебетовых и кредитных транзакций и возвращает строку "debit", "credit" или "equal".

```
function mostTransactionTypes(data) {
    let debit = 0, credit = 0;
    for (const group of data) {
```

```
        for (const item of group) {
            if (item.transaction_type == 'debit') debit++;
            else if (item.transaction_type == 'credit') credit++;
        }
    }
    if (debit > credit) return 'debit';
    if (credit > debit) return 'credit';
    return 'equal';
}
```

12. getTransactionsBeforeDate(data, date)

Возвращает транзакции, совершённые до (включительно) указанной даты. Используется та же вспомогательная функция dateToNumber.

```
function getTransactionsBeforeDate(data, date) {
    const arr = [];
    for (const group of data) {
        for (const item of group) {
            if (dateToNumber(item?.transaction_date) <= dateToNumber(date))
                arr.push(item);
        }
    }
    return arr;
}
```

13. findTransactionById(data, id)

Осуществляет поиск транзакции по уникальному идентификатору. Возвращает найденный объект или undefined.

```
function findTransactionById(data, id) {
    for (const group of data) {
        for (const item of group) {
            if (item.transaction_id == id) return item;
        }
    }
}
```

14. mapTransactionDescriptions(data)

Возвращает массив только из описаний транзакций. Применяется метод flat() для «выравнивания» вложенного массива и map() для извлечения нужного поля.

```
function mapTransactionDescriptions(data) {
    return data.flat().map(item => item?.transaction_description);
}
```

Вывод

В ходе выполнения лабораторной работы были изучены основные подходы к обработке массивов объектов в JavaScript. Реализованные функции охватывают широкий спектр операций: фильтрацию, агрегацию, поиск, группировку по датам и типам транзакций.

Были закреплены навыки:

- работы с вложенными массивами и объектами;
- применения Set для получения уникальных значений;
- сравнения дат, представленных в строковом формате;
- использования методов flat(), map(), reduce(), forEach();
- написания вспомогательных функций и их повторного использования.

Ссылка на репозиторий Git: https://github.com/maximcv246-a11y/JS-Lab3

Ответы на контрольные вопросы

1. Какие методы массивов можно использовать для обработки объектов в JavaScript?

JavaScript предоставляет богатый набор встроенных методов для работы с массивами объектов:

- forEach() — перебирает каждый элемент массива и выполняет переданную функцию. Не возвращает результата. Подходит для побочных эффектов (вывод, мутация).
- map() — создаёт новый массив, преобразуя каждый элемент через функцию-колбэк. Длина результата всегда равна длине исходного массива.
- filter() — возвращает новый массив из элементов, для которых колбэк вернул true. Используется для отбора объектов по условию.
- find() — возвращает первый элемент, удовлетворяющий условию, или undefined.
- findIndex() — аналогично find(), но возвращает индекс элемента.
- some() — возвращает true, если хотя бы один элемент соответствует условию.
- every() — возвращает true, только если все элементы соответствуют условию.
- reduce() / reduceRight() — сворачивает массив в одно значение (число, строку, объект) с помощью аккумулятора.
- sort() — сортирует массив на месте; при работе с объектами принимает функцию-компаратор.
- includes() / indexOf() — проверяют наличие значения в массиве (не объектов, а примитивов).

2. Как сравнивать даты в строковом формате в JavaScript?

Даты лучше сравнивать через объект `Date`, потому что тогда они преобразуются в числовой формат (миллисекунды) и корректно сравниваются. Также можно сравнивать строки напрямую, но только если формат `YYYY-MM-DD`, так как он правильно сортируется лексикографически.

Примеры:

```
new Date("2024-03-01") < new Date("2024-04-01") // true
"2024-03-01" < "2024-04-01" // true
```

3. В чём разница между map(), filter() и reduce() при работе с массивами объектов?

`map()` преобразует каждый элемент массива и возвращает новый массив той же длины, `filter()` отбирает элементы по условию и возвращает массив меньшего или равного размера, а `reduce()` сводит весь массив к одному значению (например сумме или объекту), накапливая результат.